W1 · CORE

Transformer Architecture

Attention Is All You Need · Vaswani et al., 2017 · arXiv:1706.03762

CORE EQUATIONS

SCALED DOT-PRODUCT ATTENTION

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Q = query, K = key, V = value, d_k = key dimension. Divide by $\sqrt{d_k}$ to prevent vanishing gradients in softmax.

MULTI-HEAD ATTENTION

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1,\ldots,\text{head}_h)W^O$$
$$\text{head}_i = \text{Attention}(QW_iQ,\ KW_iK,\ VW_iV)$$

FEED-FORWARD BLOCK

$$\text{FFN}(x) = \max(0,\ xW_1 + b_1)W_2 + b_2$$

POSITIONAL ENCODING (SINUSOIDAL)

$$PE(pos,2i) = \sin(pos / 10000^{(2i/d)})$$
$$PE(pos,2i+1) = \cos(pos / 10000^{(2i/d)})$$

ROPE (ROTARY PE — USED IN LLAMA/MISTRAL)

Apply rotation matrix $R(\theta \cdot pos)$ to Q and K

$f(x,m) = x \cdot e^{(im\theta)}$ [complex notation]

ARCHITECTURE COMPONENTS

Input embed	Token ID → dense vector (d_model dim)
Pos. encode	Add position info to embeddings
Self-attn	Each token attends to all others
Cross-attn	Decoder attends to encoder output
Layer norm	Pre-norm: applied before attn/FFN
Residual	x = x + sublayer(x) — prevents vanishing grad
d_model	512 (base), 768 (BERT), 4096 (LLaMA-7B)
n_heads	8 (base) — each head: d_model / n_heads dim
n_layers	6 encoder + 6 decoder (base GPT: 12 decoder)
FFN dim	4 × d_model (inner dimension)

COMPLEXITY

OP	TIME	SPACE
Self-attention	$O(n^2 \cdot d)$	$O(n^2)$
FFN	$O(n \cdot d^2)$	$O(d^2)$
FlashAttn	$O(n^2 \cdot d)$	O(n) — tiling

⚑ Attention is $O(n^2)$ — the quadratic bottleneck. FlashAttention reduces memory to O(n) using SRAM tiling without approximation.

ARCHITECTURE FLOW

Token IDs → Embeddings + PE → × N Encoder layers → Self-Attn + Add&Norm + FFN + Add&Norm → × M Decoder layers → Linear + Softmax → Logits

KEY INTERVIEW QUESTIONS

Why divide by $\sqrt{d_k}$ in attention?

Without scaling, dot products grow large with dimension, pushing softmax into regions with near-zero gradients. $\sqrt{d_k}$ keeps variance stable.

Why multi-head instead of single-head attention?

Different heads learn different relationship types (syntactic, semantic, positional) simultaneously. Concatenating gives richer representations than one large attention.

What does the masking in decoder self-attention do?

Causal mask prevents token i from attending to tokens j>i — enforces autoregressive generation (can only use past context).

Scaling Laws + Modern Architectures

Kaplan 2020 · Chinchilla 2022 · MoE · GQA · Mamba SSM

SCALING LAWS (KAPLAN VS CHINCHILLA)

KAPLAN 2020 — KEY FINDING

```
Loss ∝ N^(-0.076) · D^(-0.095) · C^(-0.050)
N=params, D=data tokens, C=compute
```

Kaplan: scale params faster than data. Led to GPT-3 (175B params, less data).

CHINCHILLA 2022 — REVISED FINDING

```
Optimal: N ∝ C^0.50, D ∝ C^0.50
Rule of thumb: ~20 tokens per parameter
```

Chinchilla: scale params AND data equally. Led to smaller but better-trained models (Llama-2, Mistral).

⚑ The disagreement: Kaplan used fixed data, Chinchilla optimized jointly. Chinchilla is now the standard — GPT-4 era models are all Chinchilla-optimal.

GQA: GROUPED-QUERY ATTENTION

TYPE	KV HEADS	MEMORY	QUALITY
MHA	= Q heads	High	Best
MQA	1 shared	Lowest	Drops
GQA	G groups	Medium	Near MHA

LLaMA-3 and Mistral use GQA with G=8 — best quality/memory tradeoff. KV cache scales with G, not H.

MIXTURE OF EXPERTS (MOE)

ROUTING FUNCTION

```
G(x) = softmax(x · Wg) [router]
output = Σ G(x)ᵢ · Expertᵢ(x) [top-k only]
```

Mixtral uses top-2 routing from 8 experts. Active params per token = 2 experts. Total params = 8 experts but inference cost = 2 experts.

Top-k	Typically k=2. Higher k = more compute, better quality
Load balance	Auxiliary loss prevents all tokens routing to same expert
Expert size	Each expert = standard FFN block
Mixtral	8 experts, top-2, 7B active / 47B total

MAMBA / SSM ARCHITECTURE

STATE SPACE MODEL RECURRENCE

```
h'(t) = A·h(t) + B·x(t)
y(t) = C·h(t)
Mamba: A,B,C,Δ are input-dependent (selective)
```

Key difference from S4: Mamba makes B, C, and Δ functions of the input x — "selective" SSM. This is what enables it to filter irrelevant tokens.

	TRANSFORMER	MAMBA
Complexity	$O(n^2)$	$O(n)$
Recall	Perfect (attention)	Compressed state
Long context	Expensive	Cheap
Training	Parallel	Parallel scan

Fine-Tuning: LoRA · QLoRA · DoRA · DPO

Hu 2021 · Dettmers 2023 · Liu 2024 · Rafailov 2023

LORA — LOW-RANK ADAPTATION

CORE IDEA

```
W' = W + ΔW = W + B·A
B ∈ ℝ^(d×r), A ∈ ℝ^(r×k), r ≪ min(d,k)
```

W is frozen. Only A and B are trained. At init: A ~ N(0,σ²), B = 0 (so ΔW=0 at start). Scale: α/r where α is a hyperparameter.

`r (rank)`	4–64 typical. Higher = more capacity, more memory
`alpha (α)`	Usually 2×r. Controls effective learning rate
`target`	q_proj, v_proj at minimum. All attention for best results
`params`	Adds r×(d+k) params per layer. Much less than full fine-tune

QLORA — 4-BIT LORA

KEY COMPONENTS

`NF4 quant.`	4-bit Normal Float — optimal for normally distributed weights
`Double quant`	Quantize the quantization constants — saves extra memory
`Paged optim.`	GPU→CPU memory offload for optimizer states on OOM spikes
`Memory`	7B model: ~4GB (QLoRA) vs ~14GB (bf16) vs ~28GB (full)

DORA — WEIGHT-DECOMPOSED LORA

```
W = m · (V / ‖V‖) [magnitude × direction]
DoRA trains m (scalar) and ΔV (via LoRA)
```

Separating magnitude and direction stabilizes training. Empirically outperforms LoRA at same rank. Use: peft.LoraConfig(use_dora=True)

DPO — DIRECT PREFERENCE OPTIMIZATION

RLHF OBJECTIVE (WHAT DPO REPLACES)

```
max E[r(x,y)] - β·KL[π_θ ‖ π_ref]
```

DPO LOSS (CLOSED-FORM SOLUTION)

```
L_DPO = -E[log σ(β·log(π_θ(y_w|x)/π_ref(y_w|x))
- β·log(π_θ(y_l|x)/π_ref(y_l|x)))]
```

y_w = chosen (winner), y_l = rejected (loser). No reward model needed — the policy IS the implicit reward model. β controls KL divergence penalty.

⚑ Key insight: the optimal RLHF policy can be expressed analytically in terms of log-probability ratios. DPO is exact RLHF, not an approximation.

SIMPO — REFERENCE-FREE DPO

```
L_SimPO = -E[log σ(γ/|y_w| · log π_θ(y_w|x)
- γ/|y_l| · log π_θ(y_l|x) - γ)]
```

No reference model! Uses length-normalized log-probabilities + margin γ. Simpler, often better than DPO. β=2.0, γ=0.5 are good defaults.

WHEN TO USE WHICH

METHOD	USE WHEN
`SFT only`	Domain adaptation, format learning, no preference data
`DPO`	Have preference pairs, want alignment without RL complexity
`SimPO`	No reference model available, want simpler training
`PPO`	Online learning, complex reward signals (math, code)

QUICK REFERENCE: LORA HYPERPARAMETER CHEAT SHEET (SEBASTIAN RASCHKA'S FINDINGS)

`r=4`	Classification tasks, simple format learning
`r=8`	Good default for most instruction tuning
`r=16+`	Complex domain adaptation, multimodal
`alpha`	Set to 2×r as default. Tune if training unstable
`dropout`	0.05–0.1. Set 0 for small datasets
`bias`	"none" or "lora_only" — "all" rarely helps
`lr`	2e-4 for LoRA. 1e-4 for QLoRA (more conservative)
`batch`	Per device 1–4, grad accum to reach 32–128 effective
`epochs`	1–3. Overfit risk high on small datasets

RLHF · Constitutional AI · RLAIF

InstructGPT (Ouyang 2022) · CAI (Bai 2022) · RLAIF (Lee 2023)

RLHF PIPELINE (INSTRUCTGPT)

1. **SFT:** Fine-tune base model on high-quality human demonstrations (13k prompts in InstructGPT)
2. **Reward Model:** Collect human comparisons (33k pairs). Train RM: base LLM + scalar head. Objective: r(y_w) > r(y_l)
3. **PPO:** Optimize SFT model against RM using PPO. KL penalty to prevent reward hacking: r_total = r_RM − β·KL(π_θ ‖ π_SFT)

REWARD MODEL LOSS

```
L_RM = -E[log(σ(r_θ(x,y_w) - r_θ(x,y_l)))]
```

PPO CLIPPED OBJECTIVE

```
L_CLIP = E[min(r_t·Â_t, clip(r_t, 1-ε, 1+ε)·Â_t)]
r_t = π_θ(a|s) / π_old(a|s)
```

⚑ InstructGPT found: RLHF model with 1.3B params was preferred over 175B SFT model. Alignment matters more than scale at inference.

PROCESS REWARD MODELS (PRMS)

ORM	Outcome RM — scores final answer only
PRM	Process RM — scores each reasoning step
Why PRM	Correct answer via wrong steps still fails PRM — catches shortcut reasoning
Used in	OpenAI o1/o3 — PRM guides beam search at inference

CONSTITUTIONAL AI (ANTHROPIC)

1. **Red team:** Model generates harmful responses to adversarial prompts
2. **Critique:** Model critiques its own response against a principle from the Constitution (e.g. "Is this response harmful?")
3. **Revision:** Model revises response based on its own critique
4. **SL-CAI:** Fine-tune on revised (preferred) vs original (rejected) pairs — RLAIF using AI feedback
5. **RL-CAI:** Train RM on AI-generated preferences, then apply PPO/DPO

⚑ CAI key insight: you can generate preference data at scale without human labelers by using the model itself as an annotator, guided by explicit principles.

RLAIF VS RLHF COMPARISON

	RLHF	RLAIF (CAI)
Feedback source	Humans	AI model
Cost	Expensive	Cheap — scalable
Consistency	Variable	Consistent
Bias	Human bias	Model bias
Quality	Ground truth	Approximate
Scale	Limited	Unlimited

RAG Pipeline + Evaluation

Lewis 2020 · RAGAS · HyDE · RAFT · Reranking

RAG PIPELINE FLOW

Ingest → Chunk docs (512 tok, 50 overlap) → Embed → Store in Qdrant

Query → Embed query → BM25 + Dense retrieval → Reciprocal Rank Fusion

Rerank → Cross-encoder scores top-20 → returns top-5

Generate → LLM receives [query + context chunks] → streamed response

HYDE — HYPOTHETICAL DOCUMENT EMBEDDING

1. LLM generates a **hypothetical answer** to the query
2. Embed the **hypothetical answer** (not the original query)
3. Retrieve documents similar to the hypothetical answer

HyDE works because answers and relevant documents are closer in embedding space than questions and documents. Best for open-domain QA.

RETRIEVAL TYPES — WHEN TO USE

TYPE	BEST FOR	WEAKNESS
Dense (vec)	Semantic similarity	Misses keywords
BM25 (sparse)	Keyword/entity match	No semantics
Hybrid (RRF)	General purpose	Slower
+Reranker	Precision critical	Adds latency

RAGAS EVALUATION METRICS

Faithfulness	Is the answer supported by the retrieved context? (no hallucination)
Answer relevancy	Does the answer address the actual question asked?
Context precision	What fraction of retrieved chunks were actually useful?
Context recall	Were all necessary facts present in the retrieved context?

FAITHFULNESS SCORE FORMULA

```
Faithfulness = |verified claims| / |total claims in answer|
```

Each claim in the answer is checked against the context. A fully faithful answer only makes claims supported by retrieved docs.

RAFT — RAG-ADAPTED FINE-TUNING

1. For each document chunk: generate question + gold answer
2. Include 2–3 **distractor chunks** alongside the gold chunk
3. Fine-tune model on (question, context+distractors, chain-of-thought answer)
4. Model learns to **ignore irrelevant context** — critical for production RAG

⚑ Key result: RAFT models score 15–35% higher on RAGAS faithfulness than base models + RAG. The distractors are essential — without them the model just memorizes.

BI-ENCODER VS CROSS-ENCODER

	BI-ENCODER	CROSS-ENCODER
Architecture	Separate encodings	Concat + attention
Speed	Fast (pre-index)	Slow (per pair)
Quality	Good	Best
Use	First retrieval pass	Reranking top-k

LLM Agents — ReAct · LangGraph · MCP

Yao 2022 · Wei 2022 · Tree of Thoughts 2023 · MCP 2024

REACT LOOP

- **T** **Thought:** "I need to search for the current population of India."
- **A** **Action:** search("current population of India 2024")
- **O** **Observation:** "India's population is approximately 1.44 billion."
- **T** **Thought:** "I now have the answer. I should respond."
- **F** **Final answer:** "India's population is approximately 1.44 billion."

⚑ ReAct interleaves reasoning (Thought) with acting (Action + Observation). This prevents hallucination by grounding each reasoning step in real tool outputs.

TREE OF THOUGHTS VS CHAIN OF THOUGHT

	COT	TOT
Structure	Linear chain	Tree / graph
Branching	None	k branches per step
Backtrack	No	Yes
Evaluation	End only	At each node
Best for	Simple reasoning	Planning, puzzles
Cost	Low	k× more LLM calls

AGENT FAILURE MODES

- **Hallucinated tool calls** — LLM invents tool names or args that don't exist → Fix: strict Pydantic schema validation
- **Infinite loops** — same action repeated → Fix: max_steps limit + loop detection (hash last 3 actions)
- **Context overflow** — long observation fills context → Fix: summarize observations > 500 tokens
- **Silent failures** — tool error ignored → Fix: always surface errors to LLM in Observation
- **Reward hacking** — agent games the metric → Fix: multi-dimensional eval with human spot-checks

LANGGRAPH CONCEPTS

StateGraph	Graph where each node reads/writes a shared state dict
Node	A function: (state) → state_update
Edge	Directed connection between nodes (unconditional)
Cond. edge	Routes to different nodes based on state value
Checkpointer	Persists state for multi-turn conversations
Interrupt	Pause graph for human-in-the-loop approval

MULTI-AGENT PATTERNS

Supervisor	One orchestrator routes to specialist workers. Best for clear subtask separation.
Network	Agents call each other peer-to-peer. More flexible, harder to debug.
Handoff	Sequential — agent A completes, passes to agent B. Good for pipelines.

MODEL CONTEXT PROTOCOL (MCP)

What	Standard protocol for LLM ↔ tool/server connections (Anthropic, 2024)
Components	MCP Server (exposes tools) + MCP Client (Claude, Cursor, etc.)
Transport	stdio (local) or SSE/HTTP (remote)
Tool def	JSON Schema — name, description, input_schema
Used by	Claude Desktop, Cursor, Zed, Continue, all major AI IDEs

MINIMAL MCP SERVER (PYTHON)

```
from mcp import Server, Tool
server = Server("my-rag-server")
@server.tool()
def search_docs(query: str) → str:
return rag_pipeline.query(query)
```

FlashAttention + Distributed Training

FlashAttention-2 (Dao 2023) · Megatron-LM · ZeRO · DDP

WHY STANDARD ATTENTION IS SLOW

- Standard attention materializes the full N×N attention matrix in HBM (GPU global memory)
- HBM bandwidth is ~2TB/s — reading/writing N×N is the bottleneck, not FLOPs
- For N=4096: attention matrix = $4096^2 \times 2$ bytes = 32MB per head — bandwidth limited

FLASHATTENTION — IO-AWARE TILING

1. **Tile Q, K, V** into blocks that fit in SRAM (fast on-chip memory, ~20MB)
2. **Compute attention** block by block — never write full N×N matrix to HBM
3. **Online softmax** (log-sum-exp trick) — accumulate correct softmax across blocks
4. **Recomputation** — during backward, recompute attention from Q,K,V instead of storing it

MEMORY COMPLEXITY

```
Standard: O(N²) HBM reads/writes
Flash: O(N) HBM reads/writes ← key gain
```

FLOPs are the same. The speedup comes entirely from reducing HBM bandwidth usage. On H100: ~3× faster than standard attention at N=4096.

PARALLELISM STRATEGIES

TYPE	WHAT'S SPLIT	BEST FOR
Data (DDP)	Batches across GPUs	All models — default
Tensor (TP)	Weight matrices across GPUs	Very large layers
Pipeline (PP)	Layers across GPUs	Very deep models
Sequence (SP)	Tokens across GPUs	Very long context

ZERO — MEMORY OPTIMIZATION STAGES

STAGE	WHAT'S SHARDED	MEMORY SAVING
ZeRO-1	Optimizer states	~4×
ZeRO-2	+ Gradients	~8×
ZeRO-3	+ Parameters	~64× (model size/GPUs)
ZeRO-inf	+ CPU offload	Near unlimited

ZeRO-3: each GPU holds only 1/N of parameters. Communication cost increases but enables training models that don't fit on any single GPU.

PYTORCH DDP VS FSDP

DDP	Each GPU holds full model. Gradients all-reduced. Simple. For models that fit on 1 GPU.
FSDP	Parameters sharded across GPUs (ZeRO-3 equivalent). For models > 1 GPU memory.
Grad accum	Accumulate over N steps before optimizer update. Simulates N× larger batch.
Act. checkpt	Don't store activations for backward. Recompute them. ~1.4× slower, ~3× less memory.

KEY INTERVIEW QUESTIONS

Explain why FlashAttention is faster without approximation.

It reduces HBM bandwidth usage via SRAM tiling. The bottleneck is memory bandwidth, not FLOPs. FlashAttention performs the same FLOPs but reads/writes O(N) instead of $O(N^2)$ to slow HBM.

What is activation checkpointing and when would you use it?

Instead of storing all forward activations for backward pass, recompute them on demand. Trades compute for memory. Use when GPU memory is the bottleneck — typically for large models or long sequences.

Mechanistic Interpretability

Toy Models of Superposition · SAE · Activation Steering — Anthropic

SUPERPOSITION HYPOTHESIS

> Neural networks represent more features than they have dimensions by encoding multiple features in superposition — overlapping, non-orthogonal directions. They "sacrifice" linear algebra cleanliness to store more information.

Monosemantic	One neuron → one concept (ideal, rarely found)
Polysemantic	One neuron → multiple unrelated concepts (common)
Superposition	n features stored in d dimensions where n >> d
Interference	Cost of superposition: features slightly activate each other

SUPERPOSITION MODEL (TOY MODELS PAPER)

```
x̂ = ReLU(W^T W x + b)
Loss = Σ importance(i) · (xᵢ - x̂ᵢ)²
```

$W \in \mathbb{R}^{(d \times n)}$, n > d. The model learns to pack n features into d dimensions. Rare, important features get the most "dedicated" directions.

SPARSE AUTOENCODERS (SAE)

1. **Hook** into a layer's activation (residual stream, MLP out)
2. **Encoder** maps to a larger sparse space: f(x) = ReLU(W_enc · x + b_enc)
3. **Decoder** reconstructs: x̂ = W_dec · f(x) + b_dec
4. **Loss:** reconstruction error + λ·L1(f(x)) [L1 enforces sparsity]
5. **Interpret** each SAE feature by finding its top activating tokens

ACTIVATION STEERING (ACTADD / REPE)

1. Collect activations at layer L for concept A and concept B
2. Steering vector: v = mean(act_A) - mean(act_B)
3. During inference: h_L ← h_L + α·v
4. Model behavior shifts toward concept A (or away from B)

Example: steering vector (French - English) added to residual stream causes model to generate French. Works for abstract concepts too: happiness, formality, refusal.

KEY INTERPRETABILITY CONCEPTS

Circuit	Subgraph of attention heads + MLPs implementing a specific behavior
Induction head	Two-head circuit that implements in-context learning via copying
Residual stream	The central "working memory" — each layer reads/writes to it
Logit lens	Project residual stream at each layer to vocab — track prediction formation
Probing	Train linear classifier on activations — test if concept is linearly encoded

KEY INTERVIEW QUESTIONS

Why do LLMs need superposition? Why not just have more neurons?

Compute scales with d^2, but the number of features in the world scales with experience. Superposition is a compression strategy — the model stores more concepts than neurons by accepting small interference costs for rare features.

What is the goal of a sparse autoencoder in interpretability?

To decompose polysemantic neuron activations into monosemantic features. The SAE learns a larger but sparser basis where each direction (feature) corresponds to one interpretable concept.

LLM Security + Red-Teaming

GCG Attacks · Many-Shot Jailbreaking · Prompt Injection · Defense Layers

ATTACK TAXONOMY

ATTACK	METHOD	DEFENSE
Prompt injection	Malicious instruction in user input/docs	Input classification (Prompt Guard)
Jailbreak (manual)	Role-play, hypothetical framing	Output safety filter (Llama Guard)
GCG (automated)	Optimized adversarial suffix	Perplexity filter + Prompt Guard
Many-shot	Long context with harmful examples	Context length limits + monitoring
Data extraction	Ask model to repeat training data	Output PII detection + rate limits
Indirect injection	Malicious content in retrieved docs	Sandboxed tool calls + output validation

GCG ATTACK — HOW IT WORKS

1. Start with a jailbreak prompt + random suffix tokens
2. Compute gradient of "harmful response probability" w.r.t. suffix tokens
3. Greedily swap tokens in suffix to maximize the harmful response likelihood
4. Repeat until suffix causes model to comply

GCG suffixes look like gibberish to humans (e.g. "! ! ! describing.-- Hope") but are optimized to bypass safety training. Transfers across models.

LAYERED DEFENSE ARCHITECTURE

User input → Raw text from user

Layer 1: Prompt Guard 2 → Block prompt injection attempts

Layer 2: Llama Guard 3 → Classify input safety category

LLM generates response

Layer 3: Llama Guard 3 → Classify output safety

Layer 4: Guardrails AI → PII detection + toxicity filter

Langfuse logging → Log all flagged requests for review

Safe response to user

GARAK RED-TEAMING PROBE CATEGORIES

- **jailbreak** — tests resistance to role-play and framing attacks
- **injection** — prompt injection via various delimiters and formats
- **leakage** — system prompt extraction, training data memorization
- **hallucination** — factual accuracy on known-answer questions
- **toxicity** — harmful content generation attempts
- **continuation** — completes harmful sentences from partial prompts

⚑ Target false positive rate: less than 2% on legitimate queries. A security layer blocking 5% of real user requests is broken. Measure both attack detection AND clean pass-through rate.

Production Serving + MLOps

vLLM · PagedAttention · Kubernetes · CI Evals · Cost Optimization

VLLM + PAGEDATTENTION

Problem	KV cache grows dynamically — hard to preallocate, causes fragmentation
PagedAttn	Manage KV cache like OS virtual memory — non-contiguous physical blocks
Block table	Maps logical KV positions to physical GPU memory blocks
Benefit	~3× higher throughput than HuggingFace generate() — near-zero KV waste
Continuous batching	New requests join mid-generation — GPU always fully utilised

LATENCY METRICS TO TRACK

TTFT	Time to first token. Measures prefill speed. Target: <500ms
TPOT	Time per output token. Measures decode speed. Target: <50ms
p50/p95/p99	Percentile latencies. p99 shows tail behavior under load
Throughput	Total tokens/sec across all requests. vLLM target: 500–2000 tok/s

SEMANTIC CACHING WITH REDIS

1. On query: embed question → search Redis for cosine-similar past queries
2. Cache hit (cosine > 0.95): return stored response in ~50ms
3. Cache miss: run full RAG + LLM → store result with TTL=3600s

CI EVAL PIPELINE (GITHUB ACTIONS)

PR opened → Triggers GitHub Actions workflow

`docker-compose up` → Spin up full stack for testing

`pytest tests/evals/` → Run DeepEval metric tests

Fail if score drops → faithfulness <0.8, hallucination >0.1

Pass → merge allowed

COST OPTIMIZATION STACK

TOOL	WHAT IT DOES	TYPICAL SAVING
LiteLLM	Route easy queries to cheap model	60–80% cost cut
Semantic cache	Skip LLM on repeated queries	20–40% hit rate
LLMLingua	Compress long prompts 2–4×	50–75% token cut
Batch API	Process non-urgent jobs in batch	50% price cut

KUBERNETES QUICK REFERENCE

Pod	1+ containers that share network + storage
Deployment	Manages desired replica count + rolling updates
Service	Stable network endpoint in front of pods
HPA	Auto-scales replicas based on CPU/custom metric
ConfigMap	Store non-secret config as env vars
Secret	Store API keys — mounted as env vars in pods

Interview Quick-Reference

Anthropic · OpenAI · DeepMind — from-scratch implementations + common questions

IMPLEMENT FROM SCRATCH — TIME TARGETS

TASK	TARGET TIME	DIFFICULTY
Scaled dot-product attention	<10 min	easy
Multi-head attention	<20 min	med
Layer normalization	<10 min	easy
LoRA weight update (NumPy)	<15 min	med
BPE tokenizer (merge loop)	<30 min	med
Beam search	<25 min	med
Top-k / top-p sampling	<15 min	easy
Backprop on 2-layer MLP	<30 min	hard

10 MUST-KNOW EXPLAIN-OUT-LOUD ANSWERS

1. Why does attention use $\sqrt{d_k}$ scaling?

Dot products grow with dimension, pushing softmax into low-gradient saturation regions. $\sqrt{d_k}$ keeps variance ~1 regardless of d.

2. Why is DPO equivalent to RLHF?

The optimal RLHF policy has a closed-form solution expressible in log-probability ratios. DPO directly optimizes this form, bypassing the reward model entirely.

3. How does FlashAttention achieve O(N) memory?

By tiling Q, K, V into SRAM blocks and computing attention block-by-block, it never materializes the full N×N matrix in slow HBM memory.

4. What problem does LoRA solve?

Full fine-tuning requires gradients for all parameters (~14GB for 7B model). LoRA freezes the base model and adds trainable low-rank matrices (~50MB), reducing memory 99%.

5. What is superposition in neural networks?

Storing more features than neurons by representing features as non-orthogonal directions. Trade-off: more capacity, but features interfere with each other (polysemanticity).

6. What is the difference between Kaplan and Chinchilla?

Kaplan (2020) found scale parameters faster than data. Chinchilla (2022) showed this was wrong — optimal is to scale both equally at ~20 tokens/param. GPT-3 was undertrained.

7. Why does Constitutional AI not need human labelers?

The model critiques its own outputs against written principles, generating preference data automatically (RLAIF). This scales to millions of examples at near-zero cost.

8. What is PagedAttention and why is it faster?

Manages KV cache like OS virtual memory — non-contiguous blocks, no fragmentation. With continuous batching, GPU utilization approaches 100% vs ~40% for naive serving.

9. What is the difference between a process reward model and outcome reward model?

ORM scores the final answer only. PRM scores each reasoning step — catches "right answer, wrong reasoning" which ORMs miss. Critical for math and code tasks.

10. What makes an MoE model efficient at inference?

Only top-k experts activate per token. Mixtral has 47B total params but only 12B active per forward pass — you get large model capacity at medium model compute cost.

SEBASTIAN RASCHKA'S RULE OF THUMB

⚑ When preparing for ML interviews: be able to explain every architecture from first principles in under 5 minutes. If you need notes, you don't know it yet. Practice out loud, not just in your head.

PAPER CHEAT SHEET — 5 MUST-KNOW FULL NAMES

- **"Attention Is All You Need"** — Vaswani et al., 2017, Google Brain
- **"Training Language Models to Follow Instructions with Human Feedback"** — Ouyang et al., 2022, OpenAI (InstructGPT)
- **"LoRA: Low-Rank Adaptation of Large Language Models"** — Hu et al., 2021, Microsoft
- **"Direct Preference Optimization: Your Language Model is Secretly a Reward Model"** — Rafailov et al., 2023, Stanford
- **"Toy Models of Superposition"** — Elhage et al., 2022, Anthropic